06

SKT K-뉴딜 아카데미 · AI· 자동화 기초

트리거 · 조건 · 액션 스케줄러

alert(경보)를 웹훅 수신 서버로 전송하고, 이미 전송한 alert 는 다시 보내지 않습니다.
schedule 로 정해진 간격마다 자동으로 실행합니다.

셀은 위에서 아래로 실행합니다

의존: 아래 셀이 위 셀에서 만든 변수와 함수를 사용한다는 뜻입니다.
위 셀을 실행하지 않으면 아래 셀은 실행되지 않습니다.

위 셀을 실행한 뒤 아래 셀을 실행

셀 ① `events = [...]` 실행

↓ events 가 만들어짐

셀 ② `for event in events:` 실행

정상 출력

위 셀을 건너뛰고 아래 셀을 실행

셀 ① `events = [...]` 건너뜀

↓ events 가 없음

셀 ② `for event in events:` 실행

NameError: name 'events' is not defined

> NameError 가 나오면 따옴표 안의 이름을 만든 위쪽 셀을 먼저 실행합니다.

5교시에 사용하는 용어는 다섯 개입니다

용어 (읽는 법)	한 줄 뜻
워크플로 (workflow)	정해진 순서로 실행되는 작업의 흐름
트리거 (trigger)	워크플로를 시작시키는 사건
조건 (condition)	액션을 실행할지 정하는 기준
액션 (action)	조건이 맞을 때 실행하는 작업
`requests.post()` (리퀘스츠 포스트)	POST 요청을 보내는 함수

탐지 룰은 alert(경보)를 만드는 기준입니다

탐지 룰은 로그를 보고 「이런 상황이면 alert(경보)를 만든다」고 정해 둔 기준입니다.

룰 이름 (읽는 법)	어떤 상황인가	오후 실습의 예시
brute_force (브루트 포스)	한 계정에서 로그인 실패가 반복된다	admin 계정 실패 4회
password_spraying (패스워드 스프레이잉)	한 IP 가 여러 계정에 로그인을 시도한다	한 IP 가 계정 4개에 시도
night_login (나이트 로그인)	심야 시간대에 로그인한다	03:17 에 로그인

오후에는 룰을 새로 만들지 않습니다.
룰이 만든 alert 3건을 전송하고, 중복 전송을 막고, 정해진 간격마다 실행합니다.

로그인 실패가 3회 이상이면 alert(경보)를 출력합니다

① 트리거
로그인 기록이 들어옴

→

② 조건
실패 횟수가 3회 이상인가?

→

③ 액션
alert 출력

용어	뜻	이 예시에서
트리거	시작 사건	로그인 기록이 들어옴
조건	실행 여부를 가르는 기준	실패 횟수가 3회 이상인가
액션	조건이 맞으면 할 일	alert 출력

지금까지 만든 프로그램 네 개를 세 부분으로 나눕니다

프로그램	트리거	조건	액션
log_parser.py	로그 파일을 읽었다	형식이 잘못된 줄인가	건너뛰고 기록한다
normalize_logs.py 의 룰 ①	로그가 기록되었다	로그인 실패 3회 이상인가	alert(경보)를 출력한다
api_client.py	alert 가 발생했다	처음 조회하는 IP 인가	IP 정보를 조회한다
scheduler_job.py (오늘)	정해진 시각이 되었다	처음 발생한 사건인가	alert 를 전송한다

맨 아래 줄이 오늘 오후에 만드는 프로그램입니다. 5교시에 액션, 6교시에 조건, 7교시에 트리거를 만듭니다.

룰 ①의 코드에서 트리거 · 조건 · 액션 줄을 찾습니다

```
import json

rows = json.load(open("normalized_logs.json", encoding="utf-8"))
count = {}

for row in rows:                        # 트리거 - 로그가 기록되었다
    if row["level"] == "WARN":          # 조건 - 로그인 실패인가
        user = row["user"]              # 액션 - 계정별 실패 횟수를 센다
        if user in count:
            count[user] = count[user] + 1
        else:
            count[user] = 1
```

events 는 alert(경보) 3개를 모아 놓은 목록입니다

> events[0]["rule"] 은 첫 번째 alert 의 룰 이름입니다.

```
events = [                          # alert 3건의 목록
    {"id": "brute_force:admin", "rule": "brute_force",
     "user": "admin", "count": 4},
    {"id": "password_spraying:185.220.101.34",
     "rule": "password_spraying", "ip": "185.220.101.34", "accounts": 4},
    {"id": "night_login:admin:03:17:09", "rule": "night_login",
     "user": "admin", "time": "03:17:09"},
]
print(events[0]["rule"])            # 첫 번째 alert 의 룰 이름
```

```
brute_force
```

if 조건에 맞는 alert(경보)만 액션을 실행합니다

alert 두 건 가운데 룰 이름이 night_login 인 한 건만 조건에 맞습니다.

```
events = [
    {"id": "brute_force:admin", "rule": "brute_force", "user": "admin"},
    {"id": "night_login:admin:03:17:09",
     "rule": "night_login", "time": "03:17:09"},
]

for event in events:                        # 트리거
    if event["rule"] == "night_login":      # 조건
        print(event["time"])                # 액션
```

```
03:17:09
```

`requests.post()` 는 POST 요청을 보냅니다

```
response = requests.post(URL, json=event, timeout=5)
result = response.json()
```

`requests.post(URL, …)`	**URL 로 POST 요청을 보냄. 앞에서 배운 `requests.get()` 은 GET 요청**
`json=event`	**딕셔너리 `event` 를 JSON 으로 바꿔 보냄. curl 의 `-H` 와 `-d` 가 하던 일**
`timeout=5`	**5초 안에 응답이 없으면 기다리지 않고 중단**
`response.json()`	**서버의 응답을 딕셔너리로 바꿈. 예: `{"count": 1, "status": "ok"}`**

alert(경보) 전송 액션을 send_alert() 함수로 만듭니다

> 함수로 만들면 전송 방식을 바꿀 때 `send_alert()` 한 곳만 수정합니다.

```
import requests

URL = "http://127.0.0.1:5005/webhook"

def send_alert(event):                  # alert 한 건을 전송하는 함수
    response = requests.post(URL, json=event, timeout=5)
    return response.json()

print(send_alert({"rule": "brute_force", "user": "admin"}))
```

```
{'count': 1, 'status': 'ok'}
```

curl 로 웹훅 수신 서버가 실행 중인지 확인합니다

```
curl -s -o /dev/null -w "%{http_code}" -X POST \
  -H "Content-Type: application/json" -d '{"rule":"ping"}' \
  http://127.0.0.1:5005/webhook
```

```
200
```

`-o /dev/null -w "%{http_code}"`	응답 내용 대신 상태 코드만 화면에 표시
`200` 이 출력되면	웹훅 수신 서버가 요청을 받아 정상 처리함
`000` 이 출력되면	서버가 실행 중이 아님. 노트북 맨 위 「**0.3** 준비」의 마지막 셀(`start_server`)을 다시 실행
`5005`	오후에 사용하는 웹훅 수신 서버의 포트 번호

`requests.post()` 로 같은 요청을 보냅니다

앞 장의 curl 명령과 같은 요청입니다.
`timeout=5` 는 5초 안에 응답이 없으면 기다리기를 멈추고 오류를 내라라는 뜻.

```
import requests

response = requests.post(
    "http://127.0.0.1:5005/webhook",
    json={"rule": "brute_force", "user": "admin"},
    timeout=5,                          # 5초 안에 응답이 없으면 중단
)

print(response.json()["status"])
```

```
ok
```

전송한 alert 를 received_alerts.json 에서 확인합니다

오전에 만든 웹훅 수신 서버는 요청을 받을 때마다 `received_alerts.json` 에 저장합니다.
전송한 alert(경보)의 수와 룰 이름이 파일에 있으면 전송에 성공한 것입니다.

```
import json

with open("received_alerts.json", encoding="utf-8") as f:
    received = json.load(f)

print(len(received), "건 받았습니다")     # 받은 alert 의 수
for item in received:
    print(item["rule"])                  # 룰 이름
```

5교시에 배운 것을 정리합니다

배운 것	뜻
트리거	워크플로를 시작시키는 사건. 코드에서는 `for` 줄
조건	액션을 실행할지 정하는 기준. 코드에서는 `if` 줄
액션	조건이 맞을 때 실행하는 작업. 코드에서는 `if` 안쪽 줄
`requests.post(URL, json=값)`	딕셔너리를 **JSON** 요청 본문으로 보내는 **POST** 요청
`send_alert()`	**alert(경보)**를 전송하는 액션을 함수로 만든 것. 전송 방식을 바꿀 때 한 곳만 수정한다

실습 ④ · 노트북 5교시 절 · 261002_pm_trigger_scheduler.ipynb

문제 5-1 ~ 5-9 · 도전 ⭐5-4 · ⭐5-10

① **맨 위 준비 셀을 모두 먼저 실행합니다**
· 웹훅 수신 서버가 포트 5005 에서 실행 중이어야 합니다

② **문제 5-1 ~ 5-3 — 트리거 · 조건 · 액션으로 나누기**
· 5-1 은 실행 전에 결과를 먼저 적습니다

③ **문제 5-5 ~ 5-9 — 웹훅 서버로 alert(경보) 보내기**
· 5-8 에서 **alert 세 건을 보내고**, 5-9 에서 `received_alerts.json` 으로 확인합니다

실행 중인 서버를 확인하고 종료한 뒤 다시 실행합니다

```
$ netstat -ano | grep ":5002"
  TCP    127.0.0.1:5002    0.0.0.0:0    LISTENING    18788
$ taskkill //F //PID 18788
```

Ctrl+C	터미널에서 실행한 서버를 종료. 가장 먼저 사용
netstat	network statistics 의 약자. 네트워크 연결 목록을 출력
-ano	옵션 세 개. all 모든 연결 · numeric 주소를 숫자로 · owning 프로그램 번호 표시
| grep ":5002"	앞 명령의 출력에서 :5002 가 있는 줄만 출력
taskkill	task(프로그램)를 kill(종료)하는 윈도우 명령어
//F //PID 18788	Force 강제로 · Process ID 가 18788 인 프로그램을 지정

같은 요청을 다시 보내도 결과는 한 번과 같아야 합니다

엘리베이터
5층 버튼 누름

→

다시
5층 버튼 한 번 더 누름

→

결과
목적지는 계속 5층

한 번 눌러도, 여러 번 눌러도 **목적지는 5층입니다.**
같은 요청을 반복해도 결과가 같으면 **멱등성** (idempotency · 아이뎀포턴시)**이 있습니다.**

alert(경보) 전송
응답이 늦어 같은 요청을 다시 보냄

→

결과
같은 alert 가 또 전송됨

응답이 늦으면 요청이 처리됐는지 알기 어려워 다시 보낼 수 있습니다.
이미 보낸 **alert** 가 또 전송되면 **새 alert 를 놓칠 수 있습니다.**

6교시에 사용하는 용어는 세 개입니다

용어 (읽는 법)	한 줄 뜻
사건 번호 (event ID)	사건 하나를 구분하는 고유한 값
`processed_ids.json`	우리가 만들 파일 이미 전송한 사건 번호를 저장하는 파일
`os.path.exists()` (오에스 패스 이그지스츠)	파일이 있는지 확인하는 함수

같은 alert(경보)를 반복 전송하면
실제 alert 를 놓칩니다 (멱등성 고려)

① **같은 로그 파일을 매번 재처리**
로그는 파일에 그대로 남아 있습니다. 5분 뒤에 다시 읽어도 같은 사건을 탐지합니다

② **실행할 때마다 같은 alert 를 다시 전송합니다**
alert 3건을 5분마다 전송하면 하루 288회 × 3건 = 864회입니다

③ **같은 alert 가 반복되면 담당자가 alert 를 확인하지 않게 됩니다**
새로 발생한 공격 alert 도 반복 alert 사이에서 놓칩니다

> 같은 작업을 여러 번 실행해도 결과가 한 번 실행한 것과 같아야 합니다.
> 이 성질을 멱등성이라고 합니다.

두 번 실행하면 alert(경보) 2건이 4회 전송됩니다

brute_force — admin 계정에서 로그인 실패가 반복됨 · night_login — 새벽 03:17:09 에 로그인함

```
events = [
    {"id": "brute_force:admin", "rule": "brute_force", "user": "admin"},
    {"id": "night_login:admin:03:17:09",
     "rule": "night_login", "time": "03:17:09"},
]
sent = 0
for turn in [1, 2]:                # 스케줄러가 두 번 실행되었다
    for event in events:
        sent = sent + 1            # 경보를 전송한다
print("전송 횟수:", sent)
```

```
전송 횟수: 4
```

같은 사건인지 판단하기 위해 사건마다 고유한 사건 번호(id)를 만듭니다

「룰 이름 + 계정」은 두 정보를 붙여 사건을 구별한다는 뜻입니다.

탐지 룰	사건 번호 (id)	만드는 방법
룰 ①	brute_force:admin	룰 이름 + 계정
룰 ②	password_spraying:185.220.101.34	룰 이름 + IP
룰 ③	night_login:admin:03:17:09	룰 이름 + 계정 + 로그의 시각

password_spraying — 여러 계정에 같은 비밀번호(예: Spring2026!)를 하나씩 시도하는 공격

같은 사건이면 실행할 때마다 같은 사건 번호가 만들어져야 합니다. 현재 시각을 넣으면 매번 달라집니다.

이미 전송한 사건 번호는 not in 으로 걸러냅니다

> done 은 이미 전송한 사건 번호의 리스트입니다. done 에 없는 **alert(경보)만 전송합니다.**

```
events = [
    {"id": "brute_force:admin", "rule": "brute_force", "user": "admin"},
    {"id": "night_login:admin:03:17:09",
     "rule": "night_login", "time": "03:17:09"},
]
done = ["brute_force:admin"]        # 이미 전송한 사건 번호
for event in events:
    if event["id"] not in done:     # done 에 없는 alert 만
        print("[전송]", event["id"])
```

```
[전송] night_login:admin:03:17:09
```

done 리스트를 파일에 저장합니다

done 을 변수에만 두면

1회차 — 3건 전송

↓ 종료하면 done 이 사라진다

2회차 — 3건을 또 전송

done 을 파일에 저장하면

1회차 — 3건 전송 후 파일에 저장

↓ 종료해도 파일은 남는다

2회차 — 0건 전송

저장하는 파일의 이름은 `processed_ids.json` 입니다. 이미 전송한 사건 번호가 들어 있습니다.

os.path.exists() 는 파일이 있는지 확인합니다

```
import os
os.path.exists("processed_ids.json")
```

import os	os 는 파일과 폴더를 다루는 파이썬 기본 모듈. 설치 안해도 됨
os.path.exists("파일 이름")	파일이 있으면 True, 없으면 False. 오류가 발생하지 않음
먼저 확인하는 이유	처음 실행할 때는 processed_ids.json 이 없음. 없는 파일을 open() 으로 열면 오류가 발생

load_done() 은 사건 번호를 읽고,
save_done() 은 저장합니다

```
import json
import os
def load_done():                    # 사건 번호를 파일에서 읽는다
    if os.path.exists("processed_ids.json"):
        with open("processed_ids.json", encoding="utf-8") as f:
            return json.load(f)
    return []           # 파일이 없으면 빈 리스트
def save_done(done):                # 사건 번호를 파일에 저장한다
    with open("processed_ids.json", "w", encoding="utf-8") as f:
        json.dump(done, f, ensure_ascii=False, indent=2)
```

전송한 사건 번호를 done 에 추가하고 파일에 저장합니다

```
done = load_done()
for event in events:
    if event["id"] not in done:
        print("[전송]", event["rule"])
        done.append(event["id"])    # done 에 추가
save_done(done)                     # 파일에 저장
```

done = load_done()	이미 전송한 사건 번호를 파일에서 읽음
not in done	done 에 없는 **alert(경보)만** 전송
done.append(event["id"])	전송한 사건 번호를 done 에 추가
save_done(done)	반복문이 끝난 뒤 파일에 저장

두 번째 실행에서는 0건을 전송합니다 (멱등성 생김)

실행	시작할 때 파일 상태	전송 건수
1회차	파일 없음 → 빈 리스트	3건
2회차	사건 번호 3개	0건
새 alert(경보) 1건이 추가된 뒤	사건 번호 3개	1건

시작할 때 `processed_ids.json` 을 읽습니다.
같은 코드를 다시 실행해도 같은 alert 를 전송하지 않습니다.
멱등성이 생겼습니다. 파일을 삭제하면 다시 3건을 전송합니다.

6교시에 배운 것을 정리합니다

배운 것	뜻
멱등성	같은 작업을 여러 번 실행해도 결과가 한 번 실행한 것과 같은 성질
사건 번호 (id)	사건 하나를 구분하는 고유한 값. 같은 사건이면 같은 값이다
`if event["id"] not in done:`	이미 전송한 사건 번호는 전송하지 않는다
`os.path.exists(파일)`	파일이 있는지 확인한다. 처음 실행할 때는 없다
`processed_ids.json`	이미 전송한 사건 번호를 저장하는 파일

실습 ⑤ · 노트북 6교시 절

문제 6-1 ~ 6-9 · 도전 ⭐6-4 · ⭐6-10

① **문제 6-1 ~ 6-3 — 반복 전송을 확인하고 `not in` 으로 걸러 낸다**

· 6-1 은 실행 전에 결과를 먼저 적습니다

② **문제 6-5 ~ 6-8 — 처리한 사건 번호를 파일에 저장한다**

·

6-8 이 핵심

입니다. 한 번 더 실행해 0건이면 성공입니다

③ **문제 6-9 — 새 alert(경보) 하나를 추가한다**

· 새 alert 한 건만 전송되어야 합니다

7교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	한 줄 뜻
스케줄러 (scheduler)	정해진 시각이나 간격에 작업을 실행하는 프로그램
schedule (스케줄)	실행 간격을 파이썬 코드로 지정하는 라이브러리
cron (크론)	운영체제에 있는 정기 실행 도구
cron 표현식 (크론 표현식)	분 · 시 · 일 · 월 · 요일 다섯 필드로 실행 시각을 지정하는 문자열

스케줄러 프로그램은 사람 대신 정해진 간격마다 작업을 실행합니다

사람이 직접 실행하면

03:00 — 공격 발생

↓ 아침까지 아무도 실행하지 않는다

09:00 — 출근해서 실행, 6시간 뒤에 탐지

스케줄러가 5분마다 실행하면

03:00 — 공격 발생

↓ 5분 안에 다음 실행

03:05 — 5분 뒤에 탐지, alert(경보) 전송

스케줄러는 정해진 시각이나 간격에 작업을 실행하는 프로그램입니다.
만드는 방법은 두 가지입니다 — 파이썬 라이브러리 `schedule` 과 운영체제의 `cron`.

schedule 은 등록과 확인 두 단계로 사용합니다

```
schedule.every(2).seconds.do(job)   # 등록
for i in range(6):
    schedule.run_pending()          # 확인
    time.sleep(1)
```

every(2).seconds.do(job)	job 함수를 2초마다 실행하도록 등록만 함 - 예약 목록에 추가하겠다는 의미. (이 줄을 실행하는 순간 job이 실행되는건 아니고, 이후 스케줄러가 실행 시각이 됐는지 확인할 때마다 job을 부름)
schedule.run_pending()	2초가 지났으면 job 을 실행. 이 줄이 없으면 실행되지 않음
for · time.sleep(1)	1초씩 대기하며 6번 반복해서 확인

job 을 2초마다 실행하도록 등록합니다

이 코드를 실행해도 아무것도 출력되지 않습니다. 등록만 했기 때문입니다.

```
import schedule
import time
schedule.clear()                    # 앞에서 등록한 작업을 지운다
def job():
    print("점검합니다")
schedule.every(2).seconds.do(job)   # 등록
```

schedule.clear() 를 쓰는 이유 — 셀을 다시 실행하면 앞에서 등록한 작업이 남아 있습니다.
그 상태로 또 등록하면 job 이 두 번씩 실행됩니다.

6초 동안 확인하면 job 이 두 번 실행됩니다

앞 장의 코드 아래에 이어서 적습니다.
1초 대기를 6번 반복하므로 약 6초입니다. job 은 2초마다 실행되므로 그동안 두 번 실행됩니다.

```
for i in range(6):                  # 6번 반복
    schedule.run_pending()          # 확인
    time.sleep(1)                   # 1초 대기 → 6번 × 1초 = 약 6초
```

```
점검합니다
점검합니다
```

3초만 확인하면 10분 간격 job 은 실행되지 않습니다

```
import schedule
import time
schedule.clear()
def job():
    print("점검합니다")
schedule.every(10).minutes.do(job)  # 10분마다
for i in range(3):                  # 3회 반복 = 약 3초
    schedule.run_pending()
    time.sleep(1)
print("끝")
```

끝

실제 운영에서는 while True 로 계속 확인합니다

노트북 — 정해진 횟수만 확인한다

```
for i in range(6):
```

6회 확인한 뒤 셀이 끝납니다. 다음 셀을 실행할 수 있습니다

실제 운영 — 종료할 때까지 확인한다

```
while True:
```

프로그램을 종료할 때까지 계속 확인합니다.
서버 프로그램과 같습니다

두 경우 모두 반복문 안의 코드는 같습니다.

`schedule.run_pending()` 을 호출하고 `time.sleep(1)` 로 대기합니다.

cron 표현식은 다섯 필드로 실행 시각을 지정합니다 (대략적으로 이해해도 됨)

필드 순서는 분 · 시 · 일 · 월 · 요일입니다. * 는 「모든 값」이라는 뜻입니다.

cron 표현식	* 가 아닌 필드	실행 시각
`0 6 * * *`	분 0 · 시 6	매일 오전 6시 정각
`30 2 * * *`	분 30 · 시 2	매일 오전 2시 30분
`*/10 * * * *`	분 10 간격	10분마다

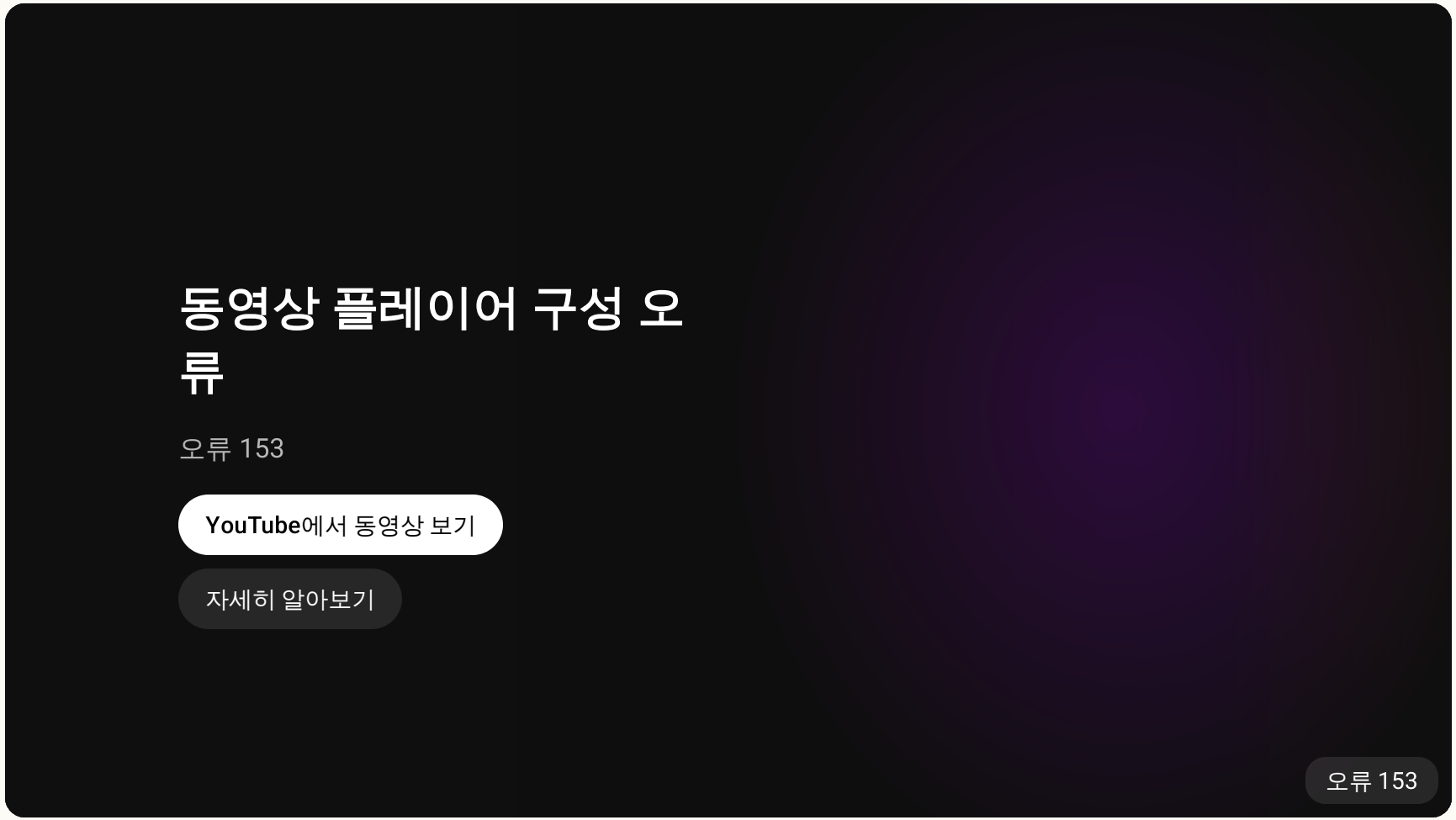
읽는 법 — * 가 아닌 필드만 읽습니다. 오늘은 이 세 가지 모양만 읽을 수 있으면 됩니다.

schedule 과 cron 은 실행하는 주체가 다릅니다

비교 기준	schedule	cron
작업을 실행하는 주체	실행 중인 파이썬 프로그램	운영체제
프로그램을 종료하면	작업이 멈춘다	정해진 시각에 다시 실행한다
윈도우 PC 에서	사용할 수 있다	사용할 수 없다 (윈도우에는 없다)

> cron 은 리눅스와 맥에 들어 있는 프로그램입니다. 윈도우에는 없어서 오늘은 schedule 로 만듭니다.

6분 영상으로 윈도우 작업 스케줄러를 봅니다



scheduler_job.py 의 트리거 · 조건 · 액션

부분	기능	배운 곳
트리거	schedule 로 --every 초마다 실행한다	7교시 · 오전 4교시 argparse
조건	로그인 실패 3회 이상이고 처음 발생한 사건이다	룰 ① · 6교시
액션	alert(경보)를 전송하고 사건 번호를 저장한다	5교시 · 6교시

새로운 문법은 없습니다. 5·6·7교시에 배운 기능을 한 파일에 모았습니다.

두 번째 실행에서는 멱등성 덕분에 새 alert(경보) 0건이 출력됩니다

`--every 5` 를 지정하면 5초마다 실행합니다. 생략하면 2초마다 실행합니다.

```
python scheduler_job.py
```

```
[전송] brute_force:admin
점검 완료 - 새 경보 1건
```

```
python scheduler_job.py
```

```
점검 완료 - 새 경보 0건
```

7교시에 배운 것을 정리합니다

배운 것	뜻
`schedule.every(2).seconds.do(job)`	2초마다 job 을 실행하도록 등록한다. job() 이 아니라 job 이라고 적는다
`schedule.run_pending()`	정해진 간격이 지났는지 확인하고, 지났으면 작업을 실행한다. 반복해서 호출한다
cron 표현식	분 · 시 · 일 · 월 · 요일 다섯 필드. `0 6 * * *` 은 매일 오전 6시

> 오후 산출물은 `scheduler_job.py` 와 `processed_ids.json` 입니다.

실습 ⑥ · 노트북 7교시 절

문제 7-1 ~ 7-9 · 도전 ⭐7-5 · ⭐7-10

① **문제 7-1 ~ 7-4 — schedule 로 일정 간격마다 실행한다**
· 7-1 은 실행 전에 결과를 먼저 적습니다

② **문제 7-6 — cron 표현식 읽기**
· 7-7 · 7-8 은 선택 문제입니다. 건너뛰고 7-9 로 가도 됩니다

③ **문제 7-9 — scheduler_job.py 완성**
· 확인 명령은 터미널에 입력합니다

오늘 만든 파일 네 개를 확인합니다

파일	하는 일	만든 교시
webhook_server.py	alert(경보)를 받는 웹훅 수신 서버	오전 4교시
test_webhook.sh	서버에 요청을 보내 확인하는 셸 스크립트	오전 3교시
scheduler_job.py	정해진 간격마다 새 alert 만 전송한다	오후 7교시
processed_ids.json	이미 전송한 사건 번호를 저장한 파일	오후 6교시

> agent_core 폴더에 네 파일이 있는지 확인하고, 깃허브에 올립니다.

남은 ⭐도전 문제를 모아 풉니다

도전	하는 일
⭐**5-10** 전송 실패 처리	서버가 꺼져 있어도 `try·except` 로 프로그램이 멈추지 않게 합니다
⭐**6-10** 사건 번호 파일 삭제	`processed_ids.json` 을 삭제하고 다시 실행합니다. 같은 **alert(경보)**를 다시 전송합니다
⭐**7-10** 탐지 룰 세 개 실행	`scheduler_job.py` 가 룰 ①②③ 을 모두 실행합니다. **1회차 3건, 2회차 0건**

> 기본 문제를 마치지 못했다면 기본 문제를 먼저 풉니다.

지금까지 만든 프로그램은 다섯 개입니다

파일	하는 일
log_parser.py	로그 파일을 읽고, 형식이 잘못된 줄은 건너뛰고 기록한다
normalize_logs.py	정규식으로 로그에서 값을 추출하고 탐지 룰을 실행한다
api_client.py	외부 API 에 IP 정보를 요청한다
webhook_server.py	alert(경보)를 받는 웹훅 수신 서버
scheduler_job.py	일정 간격으로 탐지 룰을 실행해 새 alert 만 전송한다

> 다섯 파일은 agent_core 폴더에 있고, 캡스톤 프로젝트에서 다시 사용합니다.

다음 수업은 조건을 LLM 이 판단합니다

부분	오늘	다음 수업
트리거	정해진 시각이 되었다	같다
조건	사람이 작성한 탐지 룰	LLM 이 판단한다
액션	alert(경보)를 전송한다	같다

다음 수업의 주제는 LLM 과 에이전트입니다. 트리거 · 조건 · 액션 가운데 조건을 LLM 이 판단하도록 바꿉니다.

🔔 17:30 — 수업을 마치고 오후 리추얼로 넘어갑니다. 리추얼은 20분이고 17:50 에 끝납니다.